

# Lowe's Store-Level Sales Target Model

ML-Powered Forecasting to Replace Top-Down  
Allocation

IIM Calcutta Capstone — Group 8 (APAL02)

*Live App: <https://storewisetargets.streamlit.app/>*

# The Problem: Current Approach

## Current Process

- Peanut butter spread: Uses 3-year historical sales share to allocate targets
- One-size-fits-all: Ignores local market dynamics, demographics, and competition
- Manual overrides: Planners adjust targets subjectively with limited data backing
- Inaccurate results: 25.83% of store-weeks miss targets by  $>\pm 10\%$

Impact

**25.83% plan miss rate**

# Our Solution: Supervised ML Model

## Input

- Store-level data
- Demographics
- Competition
- Historical sales

## Process

- XGBoost model
- 40 engineered features
- Rolling time split
- Store accuracy loop

## Output

- Accurate predictions
- Under/over-targeted flags
- 5-segment stratification
- Quarterly re-scoring

### Key Benefit

*Data-driven targeting enables targeted planner overrides instead of chain-wide adjustments*

# Dataset Overview

**269,412**

**Records**

store-weeks

**1,727**

**Stores**

unique locations

**194**

**Columns**

features

**0**

**Missing**

fully clean

Time Period: FY2023–FY2025 (3 fiscal years, 52 weeks each)

## Feature Composition

- 14 columns: Identity, time, target (Actual Sales USD)
- 72 columns: Demographics (population, income, housing, education)
- 108 columns: Competition (sister stores, competitor counts by radius)

# Phase 1 Feature Engineering: 35 Features in 5 Families

|                            |    |                                                       |
|----------------------------|----|-------------------------------------------------------|
| <b>Calendar</b>            | 2  | Year, Fiscal Week                                     |
| <b>Engineered Lags</b>     | 6  | lag_1, lag_52, roll_4, roll_13                        |
| <b>Store/Market</b>        | 5  | Sales Floor Size, Urbanicity, CBSA Type               |
| <b>Demand/<br/>Housing</b> | 16 | Households, Income, Housing Age, Population           |
| <b>Competition</b>         | 6  | Sister Stores, Competitor Counts, total_competitor_ta |

# The Power Features: Engineered Lag Features

*Built with group-by-store and shift() — no leakage. Strongest predictors of store sales.*

|                |                             |              |
|----------------|-----------------------------|--------------|
| <b>lag_1</b>   | shift(1)                    | <b>0.982</b> |
| <b>lag_4</b>   | shift(4)                    | <b>0.964</b> |
| <b>lag_13</b>  | shift(13)                   | <b>0.913</b> |
| <b>lag_52</b>  | shift(52)                   | <b>0.953</b> |
| <b>roll_4</b>  | shift(1).rolling(4).mean()  | <b>0.981</b> |
| <b>roll_13</b> | shift(1).rolling(13).mean() | <b>0.964</b> |

*Why lags work: Recent sales are the strongest predictor of next period sales*

# Model Architecture & Configuration

## Primary Models

- XGBoost (n\_estimators=500, lr=0.05, max\_depth=7)
- LightGBM (n\_estimators=500, lr=0.05, max\_depth=8, num\_leaves=63)
- Ridge Regression (regularization baseline)
- Naive Lag-52 baseline (year-over-year forecast)

## Training Setup

- Rolling time split: 85th percentile cutoff (temporal integrity)
- Validation set: 50 early stopping rounds
- Hyperparameter tuning: Bayesian optimization on validation WAPE
- Final holdout: Latest ~15% of fiscal weeks (FY2025 Q3–Q4)

## Success Criteria

# Model Performance Results

| Model        | WAPE | Bias % | R <sup>2</sup> | RMSE  |
|--------------|------|--------|----------------|-------|
| XGBoost      | 7.2% | -0.8%  | 0.912          | 8,340 |
| LightGBM     | 7.4% | +0.3%  | 0.908          | 8,520 |
| Ridge        | 8.8% | +1.2%  | 0.881          | 9,100 |
| Lag-52 Naive | 8.9% | -0.5%  | 0.875          | 9,200 |

## Verdict

*XGBoost achieves 7.2% WAPE, beating the success criterion (< 8%) and the naive baseline (8.9%). Model explains 91.2% of variance with minimal bias.*



# Store-Level Accuracy Loop: Classification

## Under-Targeted

*bias < -5%*

Model predicts less than actual

## Well-Calibrated

*bias  $\pm$ 5%*

Predictions align with actuals

## Over-Targeted

*bias > +5%*

Model predicts more than actual

## Per-Store Metrics Computed

- WAPE (Weighted Absolute Percentage Error)
- Bias % (directional accuracy)
- RMSE (error magnitude)
- Quarterly bias breakdown (Q1–Q4 performance by season)

## Business Insight

*Enables targeted planner overrides: Instead of blanket adjustments, fix only the under/over-targeted stores with their quarterly bias breakdown.*

# Role-of-Store Segmentation: 5 Segments

*Derived from model outputs + market features (CAGR HH, housing\_new\_share, total\_competitor\_ta)*

## High Growth

Rising demand, low competition

Invest, expand

## Growth

Moderate growth trajectory

Maintain investment

## Neutral

Stable, well-calibrated predictions

Optimize operations

## Maintain

Mature stores, stable forecast

Efficiency focus

## Defend

High competition, declining demand

Tactical response

# Interactive Streamlit Demo App

1. Dataset Summary — 269K rows, 1,727 stores
2. Phase 1 Features — 35 features in 5 families
3. Model Configuration — XGBoost, LightGBM, Ridge
4. Model Results — WAPE, Bias,  $R^2$  comparison
5. Store Accuracy Loop — per-store classification
6. Role-of-Store — 5 segments, business actions
7. AI Commentary — Gemini 2.0 Flash insights
8. Store Drill-Down — quarterly bias breakdown
9. Download Results — export CSVs
10. Upload & Score — predictions on new data

Live Application

<https://storewisetargets.streamlit.app/>

# Technical Architecture

## Python Modules

|                                         |                                                                                                    |
|-----------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>src/feature_engineering.py</code> | <code>build_phase1_features()</code> — 40 features, leakage guard                                  |
| <code>src/train_model.py</code>         | Full pipeline: <code>load</code> → <code>engineer</code> → <code>split</code> → <code>train</code> |
| <code>src/store_accuracy_loop.py</code> | Per-store metrics + Role-of-Store assignment                                                       |
| <code>app/streamlit_app.py</code>       | Interactive demo — 10 sections, 749 lines                                                          |

## Deployment

- Streamlit Cloud: <https://storewisetargets.streamlit.app/>
- Git integration: Auto-deploy from GitHub on push
- Model artifacts: XGBoost pickle in repo
- API integration: Gemini 2.0 Flash for AI commentary

# Key Findings & Recommendations

## Key Findings

- Lag features are power drivers (corr 0.91–0.98)
- XGBoost 7.2% WAPE vs 8.9% naive baseline (19% improvement)
- Store accuracy varies: 40% well-calibrated, 30% under, 30% over-targeted
- Quarterly bias reveals seasonality: Adjust Q4 for holiday demand

## Recommendations

- Deploy XGBoost for Q3/Q4 planning; monitor WAPE weekly
- Use Role-of-Store segmentation for targeted overrides
- Quarterly re-scoring: Retrain on rolling 2-year window
- Phase 2: Add price elasticity, promotions, traffic data

# Thank You

IIM Calcutta Capstone — Group 8 (APAL02)

## Questions?

*Live App: <https://storewisetargets.streamlit.app/>*